

PRACTICAL-3

AIM: To implement the Max-Heap Sort algorithm for sorting the given elements in ascending order.

Max-Heap Sort algorithm:

Max heap sort Algorithm

Input:
output:

STEP1:- Start

STEP2: Read the number of elements

STEP3: Read n elements into the array

STEP4: Build a Max-heap

- * Start from the last non-leaf node $\frac{n}{2} - 1$
- * compare each node with its left & right
- * Find the largest element
- * if the largest element is not ^{node} then swap them
- * Repeat heapify until max heap is formed

STEP5: After building the max heap, the largest element will be at $arr[0]$

STEP6: swap $arr[0]$ with the last element of the heap

STEP7: Reduce the heap size by 1.

STEP8: Apply heapify() again to restore the max heap

STEP9: Repeat step 6-8 until only one element

STEP10: the array is now sorted in ascending order

STEP11: stop

PROGRAM/CODE:

```
#include <iostream>
using namespace std;

void heapify(int arr[], int n, int i)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i)
    {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n)
{
    // Build Max Heap
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    // Heap Sort
    for (int i = n - 1; i > 0; i--)
    {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}

int main()
{
    int n;
    int arr[100];

    cout << "Enter size of array: ";
    cin >> n;
```

```
cout << "Enter array elements: ";
for (int i = 0; i < n; i++)
    cin >> arr[i];

heapSort(arr, n);

cout << "Sorted array: ";

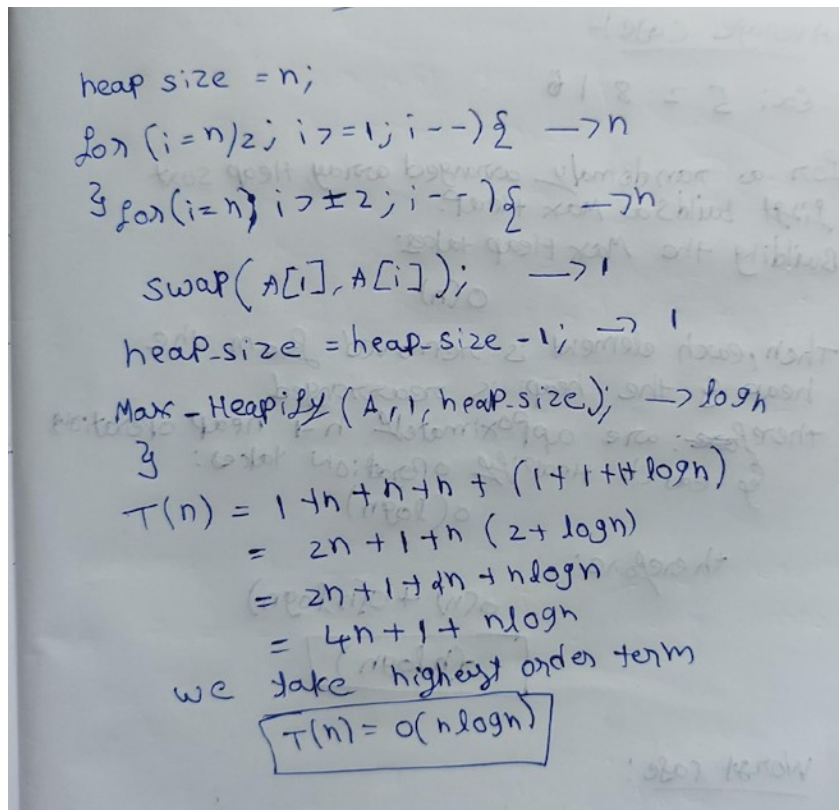
for (int i = 0; i < n; i++)
    cout << arr[i] << " ";

return 0;
}
```

OUTPUT:

```
● nandhagopal@NGKs-MacBook-Air c++ % "/Users/nandhagopal/Documents/CODING/c++/MAX-HEAP SORT"
Enter size of array: 4
Enter array elements: 22
11
44
33
Sorted array: 11 22 33 44 %
○ nandhagopal@NGKs-MacBook-Air c++ %
```

TIME COMPLEXITY:



$\text{heap size} = n;$
 $\{ \text{for } (i = n/2; i \geq 1; i--) \} \rightarrow n$
 $\{ \text{for } (i = n; i \geq 2; i--) \} \rightarrow n$
 $\text{swap}(A[i], A[1]); \rightarrow 1$
 $\text{heap-size} = \text{heap-size} - 1; \rightarrow 1$
 $\text{Max-Heapify}(A, 1, \text{heap-size}); \rightarrow \log n$
 $T(n) = 1 + n + n + n + (1 + 1 + \log n)$
 $= 2n + 1 + n(2 + \log n)$
 $= 2n + 1 + 2n + n \log n$
 $= 4n + 1 + n \log n$
 we take highest order term
 $T(n) = O(n \log n)$

1. BEST CASE:

1. Best case:-
Ex: 1 2 3 4 5
Even if array is already sorted, Heap sort still needs to build a Max Heap & perform heap operations to sort all the elements
Building the Max Heap takes: $O(n)$
for each $n-1$ elements, Heapify takes: $O(\log n)$
Therefore: $O(n) + O(n \log n)$

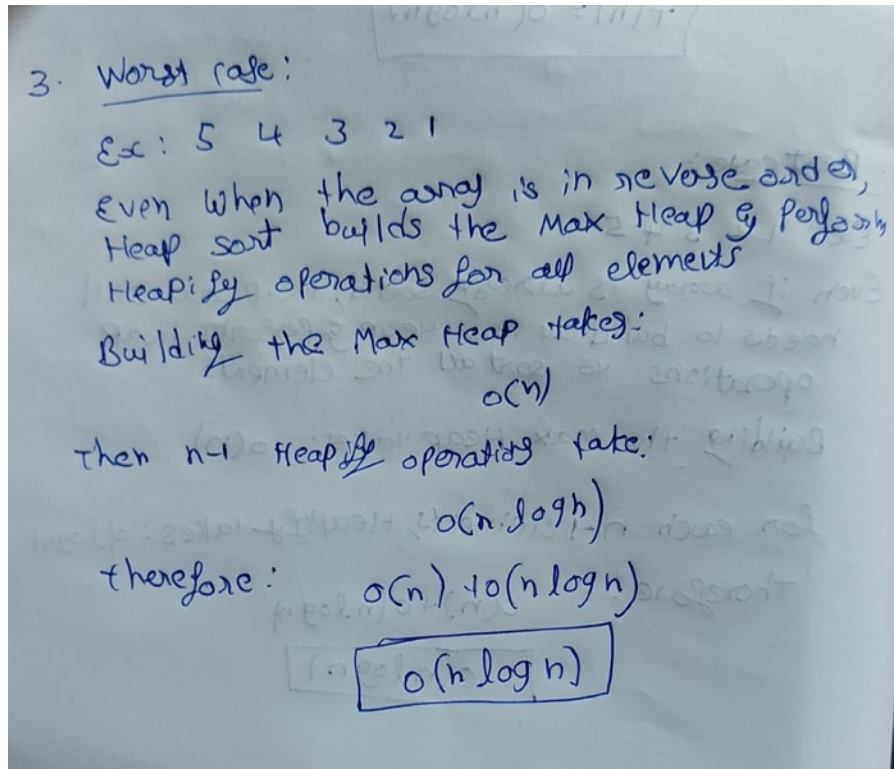
$O(n \log n)$

2. AVERAGE CASE:

1. Average case:-
Ex: 5 2 8 1 6
For a randomly arranged array, Heap sort first builds a Max Heap.
Building the Max Heap takes:
 $O(n)$
Then, each element is removed from the heap & the heap is rearranged
therefore: are approximately $n-1$ heap operations
& each Heapify operation takes:
 $O(\log n)$
therefore:
 $O(n) + O(n \log n)$

$O(n \log n)$

3. WROST CASE:



Best Case: ($O(n \log n)$)

Average Case: ($O(n \log n)$)

Worst Case: ($O(n \log n)$)

CONCLUSION:

Thus, the Max-Heap Sort algorithm was successfully implemented, and the given array elements were sorted in ascending order using the Max Heap concept.